

Techniques d'apprentissage
IFT 603-712

Méthodes à noyau et machine à vecteur de support
Par
Pierre-Marc Jodoin
/
Hugo Larochelle

1

Modélisation non linéaire

- On a vu plusieurs algorithmes basés sur des **modèles linéaires** (régression ou classification)
- Malheureusement, pas **tous les problèmes** peuvent être résolus avec un modèle linéaire
- Mis à part les réseaux de neurones, on peut obtenir des modèles non-linéaires à l'aide de **fonctions de base** non-linéaires

2

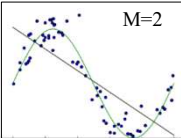
Fonctions de base polynomiales

Régression polynomiale

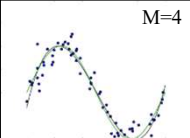
RAPPEL

Exemple tp1 : fonctions de bases polynomiales (1D) $\vec{\phi}_l(\vec{x}) = x^l$

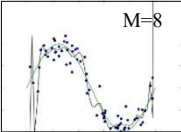
M=2



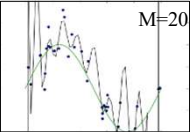
M=4



M=8



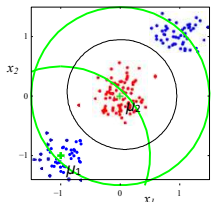
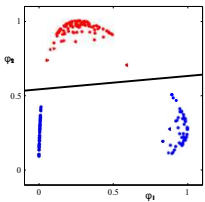
M=20



3

Fonctions de base gaussiennes

Segmentation avec noyau gaussien

Espace représenté par $\vec{x} = (x_1, x_2)^T$ Espace représenté par $\vec{\phi}(\vec{x}) = (\phi_1(\vec{x}), \phi_2(\vec{x}))^T$

2 fonctions de base gaussiennes

$$\phi_1(\vec{x}) = e^{-\frac{\|\vec{x} - \mu_1\|^2}{2\sigma^2}}$$

$$\phi_2(\vec{x}) = e^{-\frac{\|\vec{x} - \mu_2\|^2}{2\sigma^2}}$$

4

Méthodes à noyau

RAPPEL

Revenons au problème de la régression (**Maximum a posteriori** – Chap 3) :

$$\vec{w} = \arg \min_{\vec{w}} \sum_{n=1}^N \left(\vec{w}^T \vec{\phi}(\vec{x}_n) - t_n \right)^2 + \lambda \vec{w}^T \vec{w}$$

où

$$\vec{\phi}(\vec{x}) = (1, \phi_1(\vec{x}), \phi_2(\vec{x}), \dots, \phi_M(\vec{x}))$$

$$\vec{w}^T = (w_0, w_1, \dots, w_M)$$

$\uparrow$
biais

$\uparrow$
poids

5

Méthodes à noyau

Revenons au problème de la régression (**Maximum a posteriori** – Chap 3) :

$$\vec{w} = \arg \min_{\vec{w}} \sum_{n=1}^N \left(\vec{w}^T \vec{\phi}(\vec{x}_n) - t_n \right)^2 + \lambda \vec{w}^T \vec{w}$$

Si on fixe à 0 le gradient par rapport à $\vec{w}$, on observe que

Changement
de variables

$$\begin{aligned} \vec{w} &= -\frac{1}{\lambda} \sum_{n=1}^N (\vec{w}^T \vec{\phi}(\vec{x}_n) - t_n) \vec{\phi}(\vec{x}_n) \\ &= \sum_{n=1}^N a_n \vec{\phi}(\vec{x}_n) \quad \text{où } a_n = -\frac{\vec{w}^T \vec{\phi}(\vec{x}_n) - t_n}{\lambda} \in \mathbb{R} \\ &= \Phi^T \vec{a} \end{aligned}$$

6

Méthodes à noyau

Revenons au problème de la régression (**Maximum a posteriori** – Chap 3) :

$$\vec{w} = \arg \min_{\vec{w}} \sum_{n=1}^N \left(\vec{w}^T \vec{\phi}(\vec{x}_n) - t_n \right)^2 + \lambda \vec{w}^T \vec{w}$$

La solution $\vec{w}$ est une **somme pondérée des entrées** $\vec{\phi}(\vec{x}_n)$ dans l'ensemble d'entraînement

$$\vec{w} = \sum_{n=1}^N a_n \vec{\phi}(\vec{x}_n) = \Phi^T \vec{a}$$

inconnue → Connue → inconnue

7

Méthodes à noyau

$$\vec{w} = \sum_{n=1}^N a_n \vec{\phi}(\vec{x}_n) = \Phi^T \vec{a}$$

inconnue → Connue → inconnue

Idée : plutôt que d'optimiser par rapport à $\vec{w}$,
optimisons par rapport à $\vec{a}$

8

Méthodes à noyau

Partant de la loss d'origine

$$J(\vec{w}) = \sum_{n=1}^N \left(t_n - \vec{w}^T \vec{\phi}(\vec{x}_n) \right)^2 + \lambda \vec{w}^T \vec{w}$$

Si on remplace $\vec{w}$ par $\Phi^T \vec{a}$ on peut démontrer (**Bishop chap 6.1**) qu'on obtient

$$J(\vec{a}) = \vec{a}^T \Phi \Phi^T \Phi \Phi^T \vec{a} - \vec{a}^T \Phi \Phi^T \vec{t} + \vec{t}^T \vec{t} + \lambda \vec{a}^T \Phi \Phi^T \vec{a}$$

C'est la **représentation duale** de $J(\vec{w})$

9

Représentation duale

$$J(\vec{a}) = \vec{a}^T \Phi \Phi^T \Phi \Phi^T \vec{a} - \vec{a}^T \Phi \Phi^T \vec{t} + \vec{t}^T \vec{t} + \lambda \vec{a}^T \Phi \Phi^T \vec{a}$$

On peut aussi noter que $\Phi \Phi^T = K$ où $K_{nm} = k(\vec{x}_n, \vec{x}_m)$

$$k(\vec{x}_n, \vec{x}_m) = \phi(\vec{x}_n)^T \phi(\vec{x}_m)$$

Matrice de Gram

Noyau

10

Représentation Duale

Ceci se nomme un **noyau** (kernel)

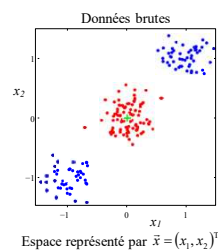
$$k(\vec{x}_i, \vec{x}_j) = \vec{\phi}(\vec{x}_i)^T \vec{\phi}(\vec{x}_j)$$

11

Matrice de Gram

La matrice de Gram est une **matrice** $N \times N$ contenant le **produit scalaire** entre chaque élément de l'ensemble d'entraînement.

Illustration en 2D



12

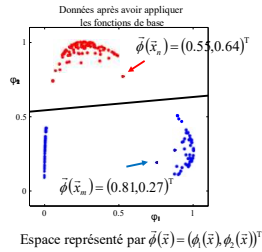
Matrice de Gram

La matrice de Gram est une **matrice $N \times N$** contenant le **produit scalaire** entre chaque élément de l'ensemble d'entraînement.

Illustration en 2D

$$K = \begin{pmatrix} k(\tilde{x}_1, \tilde{x}_1) & \dots & k(\tilde{x}_1, \tilde{x}_N) \\ \vdots & k(\tilde{x}_n, \tilde{x}_n) & \vdots \\ k(\tilde{x}_N, \tilde{x}_1) & \dots & k(\tilde{x}_N, \tilde{x}_N) \end{pmatrix}$$

$$k(\tilde{x}_n, \tilde{x}_n) = \tilde{\phi}(\tilde{x}_n)^T \tilde{\phi}(\tilde{x}_n) = (0.55 \quad 0.64) \begin{pmatrix} 0.81 \\ 0.27 \end{pmatrix} = 0.628$$



13

Représentation Duale

$$J(\vec{a}) = \vec{a}^T K K \vec{a} - \vec{a}^T K \vec{t} + \vec{t}^T \vec{t} + \lambda \vec{a}^T K \vec{a}$$

On peut démontrer qu'en **fixant à zéro la dérivée** $\nabla J(\vec{a}) = 0$ on obtient

$$\vec{a} = (K + \lambda I_N)^{-1} \vec{t}$$

Matrice $N \times N$ de Gram

Matrice Identité $N \times N$

Vecteur $N \times 1$ contenant les N cibles de l'ensemble d'entraînement

Voir preuve Bishop chap 6.1

14

Représentation Duale

Une fois $\vec{a}$ calculée, la **prédiction d'une nouvelle donnée** $\tilde{x}$ se fait comme suit

$$\begin{aligned} y(\tilde{x}) &= \vec{w}^T \tilde{\phi}(\tilde{x}) \\ &= \vec{a}^T \Phi \tilde{\phi}(\tilde{x}) \\ &= ((K + \lambda I_N)^{-1} \vec{t})^T \Phi \tilde{\phi}(\tilde{x}) \\ &= ((K + \lambda I_N)^{-1} \vec{t})^T k(\tilde{x}) \end{aligned}$$

où

$$k(\tilde{x}) = (k(\tilde{x}_1, \tilde{x}), \dots, k(\tilde{x}_N, \tilde{x}))^T$$

15

Méthode à noyau dual

Entraînement

Soient les données d'entraînement brutes $D = \{(\vec{x}_1, t_1), \dots, (\vec{x}_N, t_N)\}$

Mettre les N cibles dans un vecteur à N dimensions $\vec{t}$

Appliquer les fonctions de base à chaque donnée : $\vec{x}_n \rightarrow \vec{\phi}(\vec{x}_n) \quad \forall n$

Calculer la matrice $N \times N$ de Gram : $K(\vec{x}_n, \vec{x}_m) = \vec{\phi}(\vec{x}_n)^T \vec{\phi}(\vec{x}_m) \quad \forall n, m$

$$\vec{a} = (K + \lambda I_N)^{-1} \vec{t}$$

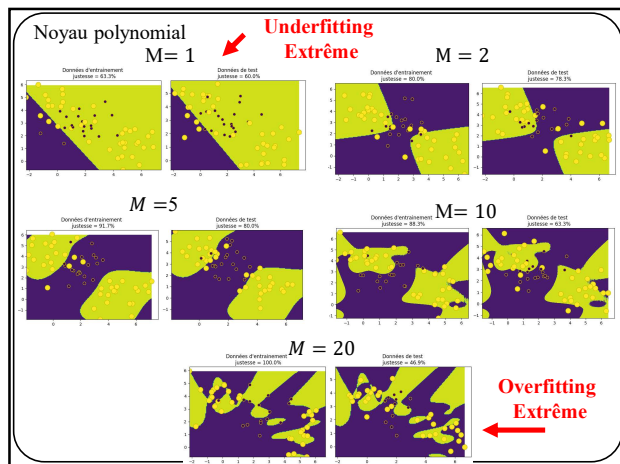
Généralisation

Appliquer les fonctions de base à la donnée à prédire : $\vec{x} \rightarrow \vec{\phi}(\vec{x})$

$$\vec{k}(\vec{x})^T = (k(\vec{x}_1, \vec{x}), \dots, k(\vec{x}_N, \vec{x})) = (\vec{\phi}(\vec{x}_1)^T \vec{\phi}(\vec{x}), \dots, \vec{\phi}(\vec{x}_N)^T \vec{\phi}(\vec{x}))$$

$$y_{\vec{a}}(\vec{x}) = \vec{k}(\vec{x})^T \vec{a}$$

16



17

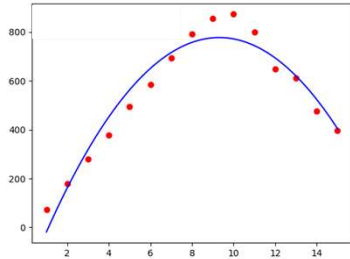
Astuce du noyau



18

Avant, revenons au TP1

Ex. fonction quadratique $x \in R$ et $\vec{\varphi}(x)^T = (1, \sqrt{2}x, x^2)$



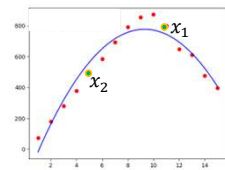
19

Astuce du noyau (*kernel trick*)

$$\begin{aligned} x_1 \in R &\rightarrow \vec{\varphi}(x_1)^T = (1, \sqrt{2}x_1, x_1^2) \\ x_2 \in R &\rightarrow \vec{\varphi}(x_2)^T = (1, \sqrt{2}x_2, x_2^2) \end{aligned}$$

$$\begin{aligned} \vec{\varphi}(x_1)^T \vec{\varphi}(x_2) &= (1, \sqrt{2}x_1, x_1^2) \begin{pmatrix} 1 \\ \sqrt{2}x_2 \\ x_2^2 \end{pmatrix} \\ &= 1 + 2x_1x_2 + x_1^2x_2^2 \end{aligned}$$

$$k(x_1x_2) = (1 + x_1x_2)^2 \quad \text{Noyau polynomial de degré 2}$$

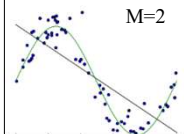


20

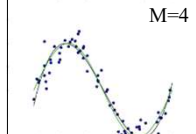
Astuce du noyau (*kernel trick*)

$k(x_1x_2)$

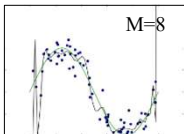
$$(1 + x_1x_2)^2$$



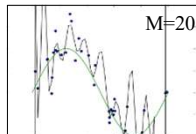
$$(1 + x_1x_2)^4$$



$$(1 + x_1x_2)^8$$



$$(1 + x_1x_2)^{20}$$



21

Autres noyaux

- **Noyau polynomial:** $k(\vec{x}_1, \vec{x}_2) = (\vec{x}_1^T \vec{x}_2 + c)^M$

➤ plus M est grand, plus le modèle a de la capacité

- **Noyau gaussien:** $k(\vec{x}_1, \vec{x}_2) = e^{-\frac{\|\vec{x}_1 - \vec{x}_2\|^2}{2\sigma^2}}$

➤ plus σ^2 est petit, plus le modèle a de la capacité

22

Grâce à l'astuce du noyau,

on définit un **noyau** $k(\vec{x}_n, \vec{x}_m)$

et non

des **fonctions de base** $\vec{\phi}(\vec{x})$



23

L'astuce du noyau

Entraînement

Soient les données d'entraînement brutes $D = \{(\vec{x}_1, t_1), \dots, (\vec{x}_N, t_N)\}$

Mettre les N cibles dans un vecteur à N dimensions $\vec{t}$

Calculer la matrice $N \times N$ de Gram : $k(\vec{x}_n, \vec{x}_m) \quad \forall n, m$

$$\vec{a} = (K + \lambda I_N)^{-1} \vec{t}$$

Généralisation

Calculer le noyau entre chaque donnée d'entraînement et $\vec{x}$: $k(\vec{x})^T = (k(\vec{x}_1, \vec{x}), \dots, k(\vec{x}_N, \vec{x}))$

$$y_{\vec{a}}(\vec{x}) = k(\vec{x})^T \vec{a}$$

24

Mêmes algos, mais **pas à pas**

Entraînement

$$D = \{(\vec{x}_1, t_1), \dots, (\vec{x}_N, t_N)\}$$

$$\vec{t} = \{t_1, \dots, t_N\}$$

Calculer la matrice $N \times N$ de Gram :

For $i=1 \dots N$ DO

For $j=1 \dots N$ DO

$$K[i][j] = k(\vec{x}_i, \vec{x}_j)$$

$$\vec{a} = (K + \lambda I_N)^{-1} \vec{t}$$

Généralisation d'une donnée $\vec{x}$

For $i=1 \dots N$ DO

$$k(\vec{x})[i] = k(\vec{x}_i, \vec{x})$$

$$y_{\vec{a}}(\vec{x}) = k(\vec{x})^T \vec{a}$$

25

Résumé

Comme au TP1

- **Problème:** $\vec{w} = \arg \min_{\vec{w}} \sum_{n=1}^N (\vec{w}^T \vec{\phi}(\vec{x}_n) - t_n)^2 + \lambda \vec{w}^T \vec{w}$
- **Paramètres :** $\vec{w} = \sum_{n=1}^N a_n \vec{\phi}(\vec{x}_n) = \Phi^T \vec{a}$ somme pondérée des entrées
- **Entraînement :** $\vec{a} = (K + \lambda I_N)^{-1} \vec{t}$ Matrice de Gram NxN
- **Prédiction:** $y(\vec{x}) = \sum_{n=1}^N k(\vec{x}, \vec{x}_n) a_n$ Noyau
- **Hyper-paramètres:**
 - λ
 - c et M pour le noyau polynomial
 - σ pour le noyau gaussien

26

Fortement recommandé pour comprendre les méthodes à noyau

Code simple pour constater comment les méthodes à noyau fonctionnent.

Comment est calculé la matrice de Gram K ?

Quelle est la dimension de K ?

Comment on entraîne un modèle à noyau?

Comment on prédit la classe d'une donnée?

Etc.

Voir code **method_noyau.zip** sur le site web du cours

27

Fortement recommandé pour comprendre les méthodes à noyau

```
def entraînement(self, x_train, t_train):
    """
    Entraîne une méthode d'apprentissage à noyau de type Maximum a
    posteriori (MAP) avec un terme d'attaché aux données de type
    "moindre carrés" et un terme de lissage quadratique
    """
    K_grame = np.array([[self.k(x, x_prime) for x_prime in x_train] for x in x_train])

    phi_mat = K_grame * self.lamb + np.identity(t_train.size)
    phi_mat_inv = np.linalg.inv(phi_mat)
    self.a = phi_mat_inv.dot(t_train)
    self.x_train = x_train
```

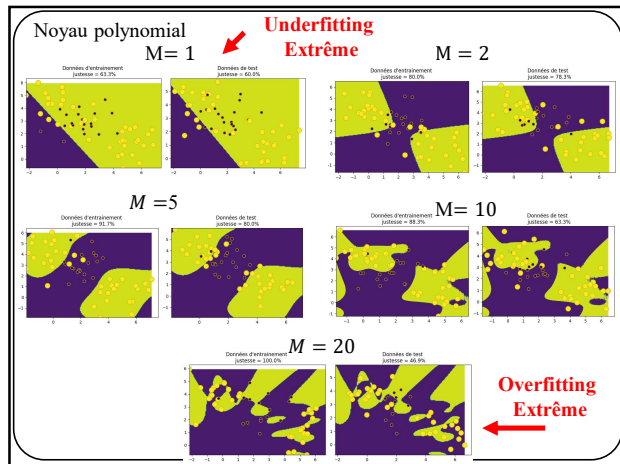
Exemple,

- noyau polynomial et RBF
- 60 données de train et test

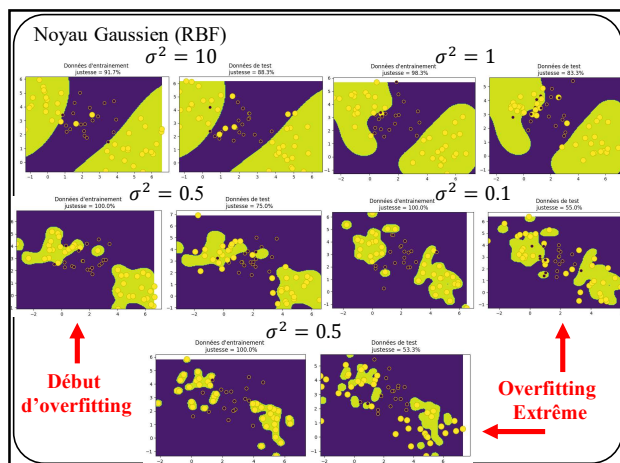
```
python main.py polynomial 60 60 -M 10
```

```
python main.py rbf 60 60 -S 0.05
```

28



29



30

Machines à vecteurs de support

(Support Vector Machines)

31

Machine à vecteur de Support

(support vector machine, SVM en anglais)

- Algorithme principalement dédié à la classification binaire
- Après l'entraînement, SVM seulement un **sous-ensemble des données d'entraînement**
- Plusieurs des a_n vont être à 0

$$\vec{w} = \sum_{n=1}^N a_n \vec{\varphi}(\vec{x}_n)$$

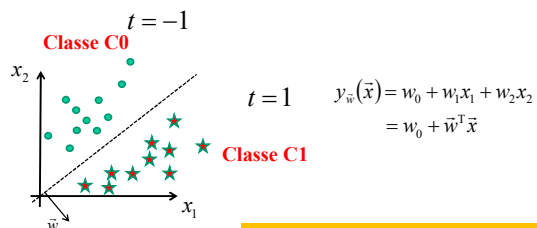
somme pondérée des données en entrée

32

32

Classification linéaire

RAPPEL



Un problème est **linéairement séparable** si on peut séparer les éléments de chaque classe avec un **hyperplan**.

33

33

Au cœur des machines à vecteurs de support figure la notion de **marge**.

34

On peut démontrer que cette équation donne la plus petite distance entre un point et une droite.

$y_w(\tilde{\phi}(\vec{x})) < 0$

$y_w(\tilde{\phi}(\vec{x})) > 0$

$t_w \frac{y_w(\tilde{\phi}(\vec{x}))}{\|\vec{w}\|}$

$\vec{x}'$

x_2

x_1

35

La **marge** est la plus **petite distance** entre la surface de séparation et les données d'entraînement

$$\text{marge} = \min_n \left(t_n \frac{y_w(\tilde{\phi}(\vec{x}_n))}{\|\vec{w}\|} \right)$$

Vecteur de support

Marge

$y > 0$

$y = 0$

$y < 0$

36

Classifieur à marge maximale

Un SVM cherche les paramètres $\vec{w}^T$ et w_0 de l'hyperplan qui **maximisent la marge**

$$\begin{aligned} & \arg \max_{\vec{w}, w_0} \{ \text{marge}(\vec{w}, w_0) \} \\ &= \arg \max_{\vec{w}, w_0} \left\{ \min_n \left(t_n \frac{y_{\vec{w}}(\vec{\phi}(\vec{x}_n))}{\|\vec{w}\|} \right) \right\} \\ &= \arg \max_{\vec{w}, w_0} \left\{ \min_n \left(t_n \frac{\vec{w}^T \vec{\phi}(\vec{x}_n) + w_0}{\|\vec{w}\|} \right) \right\} \\ &= \arg \max_{\vec{w}, w_0} \left\{ \frac{1}{\|\vec{w}\|} \min_n (t_n (\vec{w}^T \vec{\phi}(\vec{x}_n) + w_0)) \right\} \end{aligned}$$

37

37

Problème!



Il existe une **infinité de solutions** au problème de la page précédente!

La **marge est la même** si on multiplie $\vec{w}^T$ et w_0 par une **constante non nulle** (a)

$$t_n \frac{\vec{w}^T \vec{\phi}(\vec{x}_n) + w_0}{\|\vec{w}\|} = t_n \frac{\cancel{a} \vec{w}^T \vec{\phi}(\vec{x}_n) + \cancel{a} w_0}{\cancel{a} \|\vec{w}\|}$$

38

38

Solution!

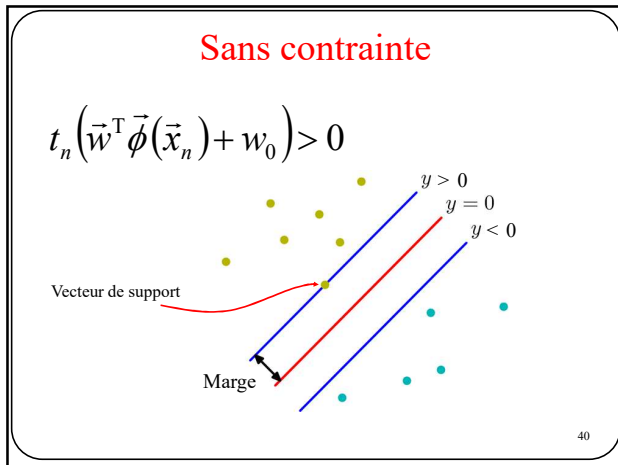


Contraindre la solution pour que les vecteurs de support

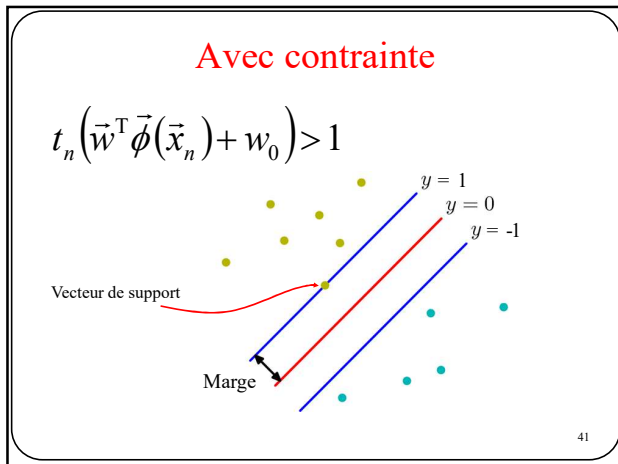
$$t_n y_{\vec{w}}(\vec{\phi}(\vec{x}_n)) = t_n (\vec{w}^T \vec{\phi}(\vec{x}_n) + w_0) = 1$$

39

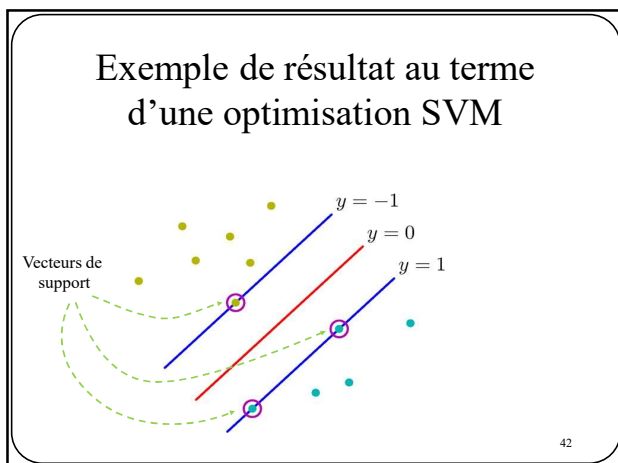
39



40



41



42

En supposant que l'ensemble d'entraînement est linéairement séparable, on a :

$$\arg \max_{\vec{w}, w_0} \left\{ \frac{1}{\|\vec{w}\|} \min_n \left(t_n \left(\vec{w}^T \vec{\phi}(\vec{x}_n) + w_0 \right) \right) \right\}$$

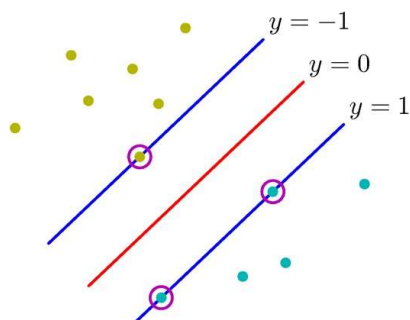
2 façons de résoudre ce problème :

- Approche primale
- Approche duale

43

Exemple

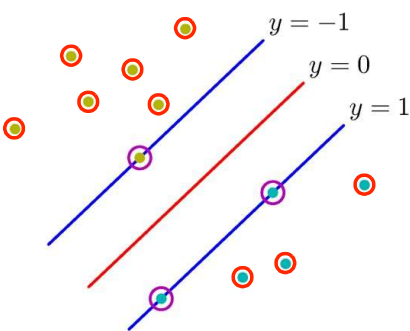
○ $t_n y(\vec{x}_n) = 1$ et $a_n \geq 0$ Vecteurs de support
○ $t_n y(\vec{x}_n) > 1$ et $a_n = 0$



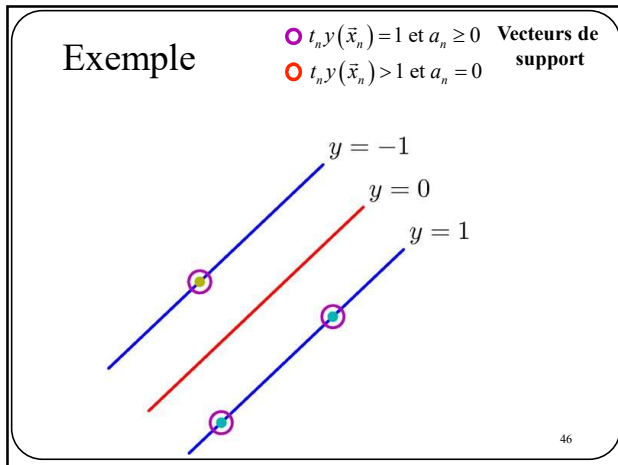
44

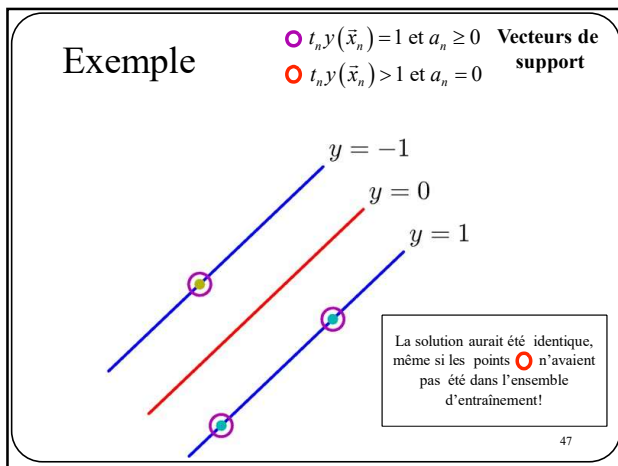
Exemple

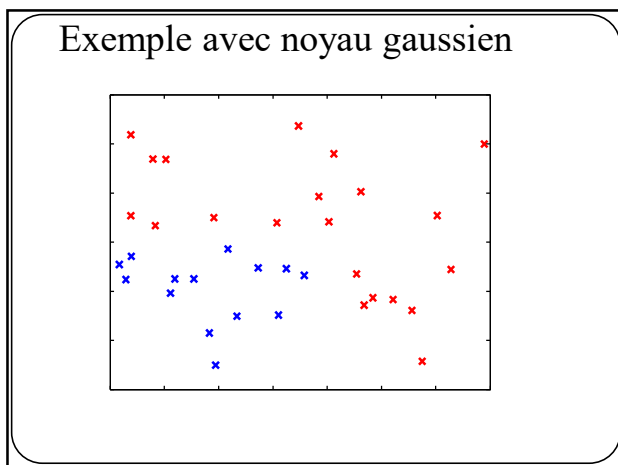
○ $t_n y(\vec{x}_n) = 1$ et $a_n \geq 0$ Vecteurs de support
○ $t_n y(\vec{x}_n) > 1$ et $a_n = 0$



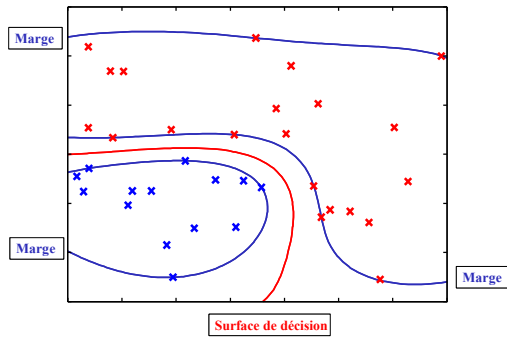
45





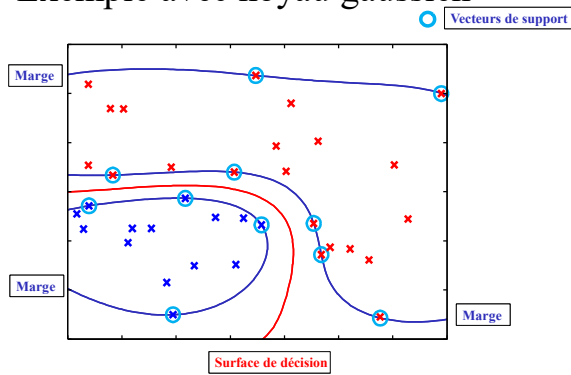


Exemple avec noyau gaussien



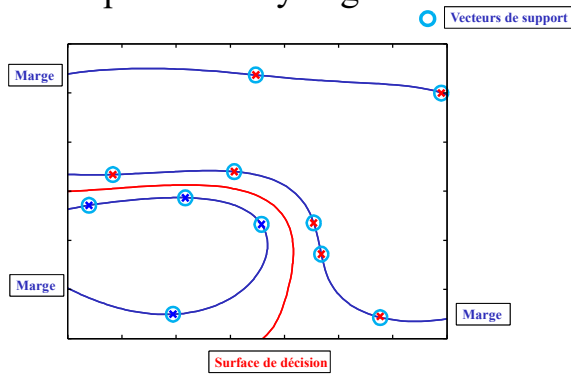
49

Exemple avec noyau gaussien



50

Exemple avec noyau gaussien



51

Données non séparables

52

52

SVM : Approche primale

RAPPEL

$$\arg \max_{\vec{w}, w_0} \left\{ \frac{1}{\|\vec{w}\|} \min_n \left(\underset{=1}{t_n \left(\vec{w}^T \vec{\phi}(\vec{x}_n) + w_0 \right)} \right) \right\}$$

Équivalent à

$$\arg \min_{\vec{w}, w_0} \left\{ \frac{1}{2} \|\vec{w}\|^2 \right\}$$

$$\text{t.q. } t_n \left(\vec{w}^T \vec{\phi}(\vec{x}_n) + w_0 \right) \geq 1 \quad \forall n$$

Ce problème d'optimisation est un **programme quadratique** pour lequel il existe de nombreuses bibliothèques

53

SVM : Approche primale

RAPPEL

$$\arg \max_{\vec{w}, w_0} \left\{ \frac{1}{\|\vec{w}\|} \min_n \left(\underset{=1}{t_n \left(\vec{w}^T \vec{\phi}(\vec{x}_n) + w_0 \right)} \right) \right\}$$

Que faire s'il y a :

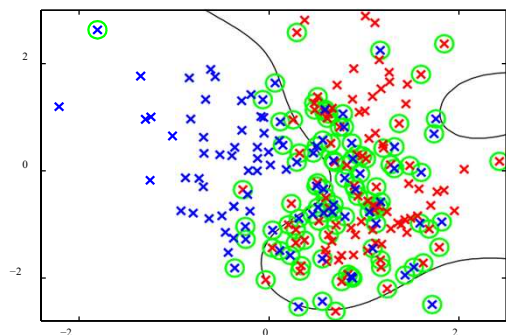
- Des données aberrantes dans l'ensemble d'entraînement?
- Si les données des 2 classes se chevauchent?

$$\text{t.q. } t_n \left(\vec{w}^T \vec{\phi}(\vec{x}_n) + w_0 \right) \geq 1 \quad \forall n$$

Ce problème d'optimisation est un **programme quadratique** pour lequel il existe de nombreuses bibliothèques

54

Exemple de classes qui se chevauchent



55

Variables de ressort (*slack variables*)

Permettre que certains exemples ne respectent pas la contrainte de marge

$$\begin{array}{l} \arg \min_{\vec{w}, w_0} \left\{ \frac{1}{2} \|\vec{w}\|^2 \right\} \\ \text{t.q. } t_n y_n (\vec{\phi}(\vec{x}_n)) \geq 1 \quad \forall n \end{array} \quad \xrightarrow{\text{Devient}} \quad \begin{array}{l} \arg \min_{\vec{w}, w_0, \xi} \left\{ \frac{1}{2} \|\vec{w}\|^2 \right\} + C \sum_{n=1}^N \xi_n \\ \text{t.q. } t_n y_n (\vec{\phi}(\vec{x}_n)) \geq 1 - \xi_n \\ \forall n, \xi_n \geq 0 \end{array}$$

Les **variables de ressort** ξ_n correspondent aux violations des contraintes de marge.

56

Variables de ressort (*slack variables*)

On va permettre que les exemples ne respectent pas la contrainte de marge

$$\begin{array}{l} \arg \min_{\vec{w}, w_0} \left\{ \frac{1}{2} \|\vec{w}\|^2 \right\} \\ \text{t.q. } t_n y_n (\vec{\phi}(\vec{x}_n)) \geq 1 \quad \forall n \end{array} \quad \xrightarrow{\text{Devient}} \quad \begin{array}{l} \arg \min_{\vec{w}, w_0, \xi} \left\{ \frac{1}{2} \|\vec{w}\|^2 \right\} + C \sum_{n=1}^N \xi_n \\ \text{t.q. } t_n y_n (\vec{\phi}(\vec{x}_n)) \geq 1 - \xi_n \\ \forall n, \xi_n \geq 0 \end{array}$$

Les **variables de ressort** ξ_n correspondent aux violations des contraintes de marge.

57

Variables de ressort (*slack variables*)

On va permettre que les exemples ne respectent pas la contrainte de marge

$$\begin{array}{l} \arg \min_{\vec{w}, w_0, \xi} \left\{ \frac{1}{2} \|\vec{w}\|^2 \right\} \\ \text{t.q. } t_n y_{\vec{w}}(\vec{\phi}(\vec{x}_n)) \geq 1 \quad \forall n \end{array} \quad \xrightarrow{\text{Devient}} \quad \begin{array}{l} \arg \min_{\vec{w}, w_0, \xi} \left\{ \frac{1}{2} \|\vec{w}\|^2 \right\} + C \sum_{n=1}^N \xi_n \\ \text{t.q. } t_n y_{\vec{w}}(\vec{\phi}(\vec{x}_n)) \geq 1 - \xi_n \\ \forall n, \xi_n \geq 0 \end{array}$$

Si ξ_n est plus grand que 1, la donnée est alors mal classée.

58

Variables de ressort (*slack variables*)

On va permettre que les exemples ne respectent pas la contrainte de marge

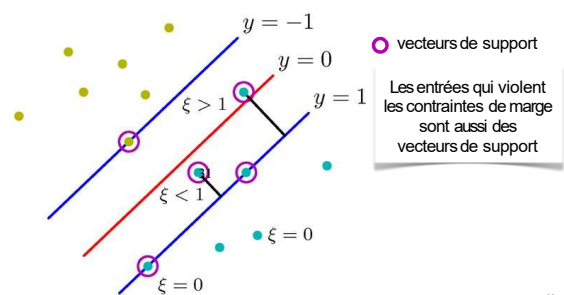
$$\begin{array}{l} \arg \min_{\vec{w}, w_0, \xi} \left\{ \frac{1}{2} \|\vec{w}\|^2 \right\} \\ \text{t.q. } t_n y_{\vec{w}}(\vec{\phi}(\vec{x}_n)) \geq 1 \quad \forall n \end{array} \quad \xrightarrow{\text{Devient}} \quad \begin{array}{l} \arg \min_{\vec{w}, w_0, \xi} \left\{ \frac{1}{2} \|\vec{w}\|^2 \right\} + C \sum_{n=1}^N \xi_n \\ \text{t.q. } t_n y_{\vec{w}}(\vec{\phi}(\vec{x}_n)) \geq 1 - \xi_n \\ \forall n, \xi_n \geq 0 \end{array}$$

La constante $C > 0$ est un hyper-paramètre

- Plus C est petit, plus on permet des données mal classées

59

Exemple (variables de ressort)



60

60

Variables de ressort – représentation **primale**

$$\begin{aligned} \arg \min_{\vec{w}, w_0, \xi} & \left\{ \frac{1}{2} \|\vec{w}\|^2 \right\} + C \sum_{n=1}^N \xi_n \\ \text{t.q. } & t_n y_{\vec{w}}(\phi(\vec{x}_n)) \geq 1 - \xi_n \\ & \forall n, \xi_n \geq 0 \end{aligned}$$

61

61

Variables de ressort – représentation **primale**

$$\begin{aligned} \arg \min_{\vec{w}, w_0, \xi} & \left\{ \frac{1}{2} \|\vec{w}\|^2 \right\} + C \sum_{n=1}^N \xi_n \\ \text{t.q. } & \xi_n \geq 1 - t_n y_{\vec{w}}(\phi(\vec{x}_n)) \\ & \forall n, \xi_n \geq 0 \end{aligned}$$

62

62

Variables de ressort – représentation **primale**

$$\begin{aligned} \arg \min_{\vec{w}, w_0, \xi} & \left\{ \frac{1}{2} \|\vec{w}\|^2 \right\} + C \sum_{n=1}^N \xi_n \\ \text{t.q. } & \xi_n \geq 1 - t_n y_{\vec{w}}(\phi(\vec{x}_n)) \\ & \forall n, \xi_n \geq 0 \end{aligned}$$

63

63

Variables de ressort – représentation primale

$$\arg \min_{\vec{w}, w_0} \frac{1}{2} \|\vec{w}\|^2 + C \sum_{n=1}^N \max(0, 1 - t_n y_{\vec{w}}(\phi(\vec{x}_n)))$$

Forme similaire à celle présentée au chapitre sur la segmentation linéaire!

64

64

Même forme qu'au chapitre 4!

$$\arg \min_{\vec{w}, w_0} \sum_{n=1}^N \max(0, 1 - t_n y_{\vec{w}}(\phi(\vec{x}_n))) + \lambda \|\vec{w}\|^2$$

Constante
Fonction de perte (Hinge loss)
Régularisation ($\lambda = 1/2C$)

Solution obtenue par **descente de gradient**

65

65

Résumé (SVM sans noyau - primal)

• **Modèle:** $y_{\vec{w}}(\phi(\vec{x}_n)) = \vec{w}^T \phi(\vec{x}_n) + w_0$

• **Problème :**

$$\arg \min_{\vec{w}, w_0, \xi} \frac{1}{2} \|\vec{w}\|^2 + C \sum_{n=1}^N \xi_n$$

t.q. $\xi_n \geq 1 - t_n y_{\vec{w}}(\phi(\vec{x}_n))$
 $\forall n, \xi_n \geq 0$

• **Hyper-paramètres:** C

• **Entraînement :** résoudre programme quadratique

66

Résumé (SVM sans noyau - primal)

• **Modèle:** $y_{\vec{w}}(\phi(\vec{x}_n)) = \vec{w}^T \phi(\vec{x}_n) + w_0$

• **Problème :**

$$\begin{array}{ll} \arg \min_{\vec{w}, w_0, \xi} & \frac{1}{2} \|\vec{w}\|^2 + C \sum_{n=1}^N \xi_n \\ \text{t.q.} & \xi_n \geq 1 - t_n y_{\vec{w}}(\phi(\vec{x}_n)) \\ & \forall n, \xi_n \geq 0 \end{array}$$

• **Hyper-paramètres:** C

• **Entraînement :** descente de gradient

$$\arg \min_{\vec{w}, w_0} \sum_{n=1}^N \max(0, 1 - t_n y_{\vec{w}}(\phi(\vec{x}_n))) + \lambda \|\vec{w}\|^2$$

67

Résumé (SVM avec noyau - dual)

• **Modèle:** $y_{\vec{w}}(\phi(\vec{x}_n)) = \vec{w}^T \phi(\vec{x}_n) + w_0$

• **Problème :** $\arg \min_a \sum_{n=1}^N a_n - \frac{1}{2} \sum_{n=1}^N \sum_{m=1}^N a_n a_m t_n t_m k(\vec{x}_n, \vec{x}_m)$
 t.q. $C \geq a_n \geq 0$ et $\sum_{n=1}^N a_n t_n = 0$

Plusieurs des a_n seront à 0

• **Hyper-paramètres:** C

• **Entraînement :** programme quadratique

• **Prédiction:** $y(\vec{x}_n) = \sum_{n=1}^N a_n t_n k(\vec{x}_n, \vec{x}) + w_0$

Seuls les vecteurs de support vont voter! Noyau

68
